



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS CRATEÚS
SISTEMAS DE INFORMAÇÃO

Disciplina: Algoritmos e Complexidade Computacional

Professor: Rafael Martins Barros

Trabalho Prático 02

Árvores Binárias de Busca

Autores:

Carlos Eduardo de Almeida Coutinho

Maria Fernanda Vasconcelos Nunes

Conteúdo

1	Introdução	2
2	Fundamentação Teórica	2
2.1	Árvore Binária de Busca	3
2.2	Árvore AVL	4
2.3	Árvore Rubro-Negra	5
3	Método Experimental	6
4	Resultados	8
4.1	Métricas Estruturais: Altura da Árvore	8
4.2	Custo Estrutural: Quantidade de Rotações	9
4.3	Tempo de Execução: Inserção	10
4.4	Tempo de Execução: Busca	10
4.5	Tempo de Execução: Remoção	11
5	Discussão	11
5.1	Análise da Árvore Binária de Busca (BST)	12
5.2	Análise da Árvore AVL	12
5.3	Análise da Árvore Rubro-Negra LLRB	13
6	Conclusão	13
7	Uso de ferramentas de IA	14

1 Introdução

O gerenciamento eficiente de conjuntos dinâmicos de dados é um dos desafios centrais no projeto e análise de algoritmos. Aplicações modernas exigem que operações fundamentais, como a busca, inserção e remoção de chaves, sejam executadas de maneira rápida e escalável, mesmo sob grandes volumes de informação.

Embora estruturas lineares apresentem limitações óbvias de desempenho em pior caso, as árvores binárias de busca surgem como uma alternativa promissora ao dividirem o espaço de busca de forma hierárquica. No entanto, a eficiência dessas estruturas está intrinsecamente atrelada à sua topologia e à ordem de entrada dos dados. Diante da possibilidade de degeneração linear das árvores simples, o estudo e a implementação de mecanismos automatizados de balanceamento tornam-se indispensáveis para garantir a previsibilidade e a eficiência assintótica dos sistemas.

O problema fundamental abordado neste trabalho reside na vulnerabilidade da Árvore Binária de Busca clássica diante de entradas previamente ordenadas ou fortemente enviesadas, cenário no qual o custo computacional de suas operações diverge do caso médio ideal $O(\log n)$ e atinge o limite linear $O(n)$ [4]. Essa imprevisibilidade compromete a viabilidade da estrutura em cenários de alta performance.

Dessa forma, este trabalho tem como objetivo principal implementar, analisar e comparar experimentalmente o desempenho de três estruturas distintas de árvores de busca: a Árvore Binária de Busca (BST) sem qualquer mecanismo de autobalanceamento, a Árvore AVL com suas operações de rebalanceamento dinâmico focadas no fator de balanceamento de suas subárvores, e a Árvore Rubro-Negra Caída para a Esquerda (LLRB) focada na posição dos links vermelhos e manutenção de suas propriedades. Para atingir o objetivo central deste trabalho, pretende-se realizar testes controlados com diferentes volumetrias e perfis de inserção com dados aleatórios e dados sequenciais para mensurar empiricamente o impacto prático das operações de rebalanceamento no tempo de execução e na altura final de cada árvore, validando os conceitos teóricos de complexidade computacional estudados.

2 Fundamentação Teórica

Esta seção apresenta os conceitos fundamentais para a compreensão das estruturas de dados apresentadas neste trabalho.

2.1 Árvore Binária de Busca

Uma Árvore Binária de Busca (BST - *Binary Search Tree*) pode ser definida como um grafo direcionado que possui um único nó raiz. Cada nó na estrutura possui dois nós descendentes, denominados de nós filhos, um imediatamente à esquerda e outro à direita. A propriedade central que define uma árvore de busca e a diferença de uma árvore binária comum é a sua regra de ordenação na qual, para qualquer nó, todas as chaves armazenadas na subárvore à esquerda são estritamente menores que a chave do nó pai, enquanto a subárvore à direita é responsável por armazenar todas as chaves maiores que o pai. É essa lógica que permite que a estrutura seja utilizada para buscas rápidas, simulando o comportamento de uma busca binária em um conjunto de dados dinâmico [4].

O comportamento dinâmico de uma árvore binária de busca é sustentado por três operações essenciais: **busca**, **inserção** e **remoção** de elementos. A operação de busca inicia-se obrigatoriamente no nó raiz. O valor da chave pretendida é comparado com a chave do nó corrente; se houver equivalência, a operação termina com sucesso. Caso a chave procurada seja estritamente menor que a do nó atual, o algoritmo ramifica-se para a subárvore esquerda; se for maior, direciona-se para a subárvore direita. Este fluxo estende-se recursivamente até que o elemento seja localizado ou que se atinja um nó vazio (nulo), o que sinaliza a ausência do dado na estrutura. A inserção replica este percurso de pesquisa e, ao alcançar o ponto de terminação nulo correspondente, aloca um novo nó e estabelece o novo vínculo de encadeamento.

Dado que a exclusão não pode, sob hipótese alguma, romper a integridade da ordenação do sistema, a remoção de um nó introduz uma complexidade estrutural superior. Esta operação divide-se em três cenários distintos baseados no grau do nó a ser eliminado: **nó sem filho**, a eliminação é direta apenas substituindo o valor presente por nulo; **nó com apenas um filho**, a referência guardada por seu nó pai passa a apontar para o único filho do nó a ser removido, removendo o elemento referencialmente; **nó com dois filhos**, o nó a ser removido deve ser substituído por seu sucessor ordenado imediato (menor elemento na subárvore direita) ou predecessor (maior elemento na subárvore esquerda) [4].

No entanto, esta estrutura apresenta riscos quanto ao seu balanceamento topológico, os quais, dependendo da ordem de inserção dos elementos, com dados previamente ordenados de forma crescente ou decrescente, a árvore tende a degenerar-se por completo e apresentar uma estrutura linear semelhante a uma lista encadeada com suas operações apresentando

complexidade de tempo, no pior caso, $O(n)$ [4]. Justificando, dessa forma, a necessidade de métodos de rebalanceamento dinâmicos, como os das estruturas subsequentes.

2.2 Árvore AVL

Uma árvore AVL é definida estruturalmente como uma BST na qual, para cada nó individual, a diferença entre as alturas de suas subárvores esquerda e direita, denominada Fator de Balanceamento (FB), deve pertencer estritamente ao conjunto $\{-1, 0, 1\}$ [1]. Matematicamente, o fator de balanceamento de um nó p é expresso pela Equação 1:

$$FB(p) = h(p.esq) - h(p.dir) \quad (1)$$

Onde $h(p.esq)$ e $h(p.dir)$ representam as alturas das subárvores à esquerda e à direita do nó p , respectivamente. Caso o valor absoluto desse fator exceda a unidade ($|FB(p)| > 1$), a árvore é considerada desbalanceada, ativando imediatamente os mecanismos de correção estrutural.

A principal melhoria introduzida pela árvore AVL em relação à árvore binária de busca clássica consiste na garantia assintótica de desempenho em cenários de pior caso. Ao impor um limite estrito à diferença de altura entre as ramificações, a estrutura impede matematicamente o fenômeno da degeneração linear. Consequentemente, as operações de busca, inserção e remoção deixam de oscilar até o limite linear de $O(n)$, passando a ser executadas sob uma complexidade de tempo estritamente logarítmica, $O(\log n)$ [1].

Para sustentar essa invariante de balanceamento ao longo de modificações dinâmicas, a árvore AVL introduz operações de rebalanceamento dinâmico mapeadas a partir do ponto onde ocorreu a violação do fator de conformidade. Essas operações consistem em transformações geométricas locais na topologia dos ponteiros, denominadas **rotações**. Dependendo da geometria do desbalanceamento provocado por uma inserção ou remoção, aplicam-se quatro tipos de rotações fundamentais:

- **Rotação Simples à Direita (LL):** Aplicada quando um nó torna-se pesado à esquerda ($FB = 2$) devido a uma inserção na subárvore esquerda de seu filho esquerdo. O filho esquerdo ascende à posição de nova raiz local, e o pai original torna-se seu filho direito.
- **Rotação Simples à Esquerda (RR):** Executada de forma simétrica à anterior,

quando um nó torna-se pesado à direita ($FB = -2$) devido a uma inserção na subárvore direita de seu filho direito.

- **Rotação Dupla à Direita (LR):** Ocorre quando um nó apresenta $FB = 2$, mas seu filho esquerdo possui $FB = -1$ (pesado à direita). Consiste em realizar primeiro uma rotação simples à esquerda no filho esquerdo, convertendo o cenário em um desbalanceamento puramente à esquerda (LL), seguida por uma rotação simples à direita no nó pai desbalanceado.
- **Rotação Dupla à Esquerda (RL):** Acionada quando um nó apresenta $FB = -2$ e seu filho direito possui $FB = 1$. Realiza-se uma rotação simples à direita no filho direito, seguida de uma rotação simples à esquerda no nó pai.

2.3 Árvore Rubro-Negra

A árvore rubro-negra clássica surgiu a partir das Árvores B Binárias Simétricas [2] e estruturalmente, trata-se de uma árvore binária de busca cujos nós armazenam um bit adicional de informação correspondente à sua cor: vermelho ou preto [3]. Para manter a árvore genericamente balanceada, o sistema deve respeitar rigidamente um conjunto de cinco propriedades fundamentais:

1. Todo nó é explicitamente colorido como vermelho ou preto.
2. A raiz da árvore é obrigatoriamente preta.
3. Todas as folhas externas (nós nulos) são consideradas pretas.
4. Se um nó é vermelho, ambos os seus nós filhos devem ser pretos (impedindo a ocorrência de dois nós vermelhos consecutivos em um mesmo caminho).
5. Todo caminho direcionado de um nó arbitrário até qualquer uma de suas folhas descendentes deve conter exatamente o mesmo número de nós pretos (propriedade conhecida como *altura preta*).

Embora garanta operações estáveis em tempo logarítmico $O(\log n)$, a árvore rubro-negra clássica é historicamente reconhecida por sua complexidade de implementação. O algoritmo tradicional exige o tratamento de inúmeros casos e subcasos simétricos durante as etapas de inserção e remoção [3].

Para sanar essa limitação prática, Robert Sedgwick introduziu a variante denominada Árvore Rubro-Negra Caída para a Esquerda (LLRB - *Left-Leaning Red-Black Tree*) [5]. A LLRB impõe uma restrição geométrica adicional e estrita: todos os links vermelhos devem, obrigatoriamente, inclinar-se para a esquerda. Essa modificação padroniza a estrutura de modo que ela passe a representar uma correspondência exata com uma Árvore 2-3. Sob essa nova regra, um 3-nó da árvore 2-3 (que contém duas chaves e três nós filhos) é universalmente representado na LLRB por dois nós binários conectados por um link vermelho posicionado exclusivamente à esquerda [5].

Essa padronização simplifica drasticamente a verificação e a aplicação das operações de rebalanceamento dinâmico. Em vez de gerenciar uma matriz complexa de estados condicionais, o rebalanceamento da LLRB passa a ser resolvido de maneira linear e sequencial ao inspecionar recursivamente três condições simples no caminho de retorno da inserção:

- **Filho direito vermelho e filho esquerdo preto:** Esta configuração viola a invariante *left-leaning*. Aplica-se uma rotação simples à esquerda para corrigir a inclinação do link vermelho.
- **Filho esquerdo vermelho e neto esquerdo também vermelho:** Configuração que gera dois nós vermelhos consecutivos verticalmente. Aplica-se uma rotação simples à direita no nó pai para readequar a topologia.
- **Ambos os filhos (esquerdo e direito) vermelhos:** Representa o equivalente a um transbordo (*overflow*) de um nó em uma árvore 2-3. Resolve-se executando uma inversão de cores (*color flip*), transformando ambos os filhos em pretos e ascendendo a cor vermelha para o nó pai.

3 Método Experimental

Os experimentos foram executados em um ambiente Linux, utilizando a distribuição CachyOS, baseada no Arch Linux. O notebook utilizado possui um processador Ryzen 5 5500U com 18GB de RAM, e todos os testes foram realizados com o equipamento conectado à energia para evitar perda de desempenho. A linguagem utilizada para a implementação das estruturas foi o Java, escolhida devido à familiaridade dos alunos com

ela, bem como pelo seu bom desempenho na execução de algoritmos. Além disso, optou-se pelo Python para o desenvolvimento do script de geração dos gráficos de resultados, dada a facilidade oferecida pela biblioteca `matplotlib`.

Foram implementadas a Árvore Binária de Busca, a árvore AVL e a Árvore Rubro-Negra (LLRB), utilizando como base as implementações contidas nos slides do professor e no livro Algorithms [5].

Optou-se pela utilização de vetores em memória para os testes em vez da leitura de arquivos externos. Como o trabalho visava avaliar estritamente o desempenho dos algoritmos em tempo de execução, número de rotações e altura da árvore, não mensurando a complexidade de espaço, essa abordagem evita o custo computacional atrelado às operações de I/O em disco. Assim, garante-se a extração do desempenho máximo das estruturas de forma mais eficiente e simples de implementar.

Os tamanhos das instâncias testadas foram:

- 1000 elementos
- 5000 elementos
- 10000 elementos
- 50000 elementos
- 100000 elementos

Conforme requisitado, cada estrutura foi testada 10 vezes para cada um dos seguintes cenários:

- Inserção aleatória
- Inserção ordenada crescente
- Inserção ordenada decrescente
- Busca aleatória (abrangendo elementos presentes e ausentes)
- Remoções aleatórias

O projeto foi dividido em seis arquivos principais. Os arquivos `ArvoreBinariaBusca.java`, `AVL.java` e `LLRB.java` implementam, respectivamente, a Árvore Binária de Busca, a

árvore AVL e a Árvore Rubro-Negra. As três estruturas implementam a interface `OperacoesArvores.java`, que concentra todas as funções inerentes às árvores. O arquivo `Main.java` é responsável pela criação dos vetores de teste, execução completa dos cenários e medição das estatísticas, gerando ao final o arquivo `resultados.csv`. Por fim, o script `plotar_graficos.py` utiliza a biblioteca `matplotlib` para gerar os gráficos com base nos resultados obtidos nos experimentos.

4 Resultados

Abaixo estão apresentados os resultados obtidos a partir das métricas definidas, englobando tanto os cenários de entradas aleatórias quanto os casos adversariais (entradas ordenadas crescentes e decrescentes). Para todas as métricas temporais, os gráficos apresentam o tempo médio de execução em milissegundos (ms) acompanhado de barras de desvio padrão. Vale ressaltar que a execução da rotina completa de testes levou aproximadamente 37 minutos, exigindo a alocação de 100MB de memória de pilha (`-Xss100m`) na Máquina Virtual Java para evitar o erro de *StackOverflowError* causado pela recursão excessiva nas instâncias degeneradas.

4.1 Métricas Estruturais: Altura da Árvore

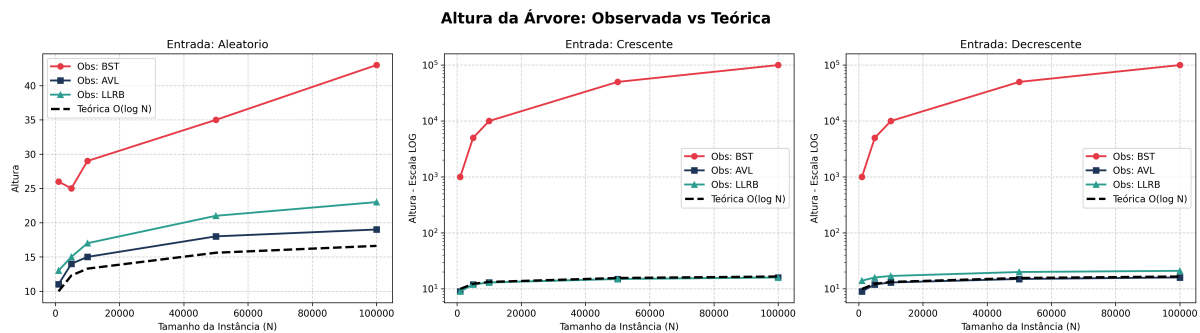


Figura 1: Altura da Árvore: Observada vs Teórica

Analisando a Figura 1, observa-se que em cenários de entrada aleatória (gráfico à esquerda), a própria aleatoriedade atua como um fator natural de balanceamento. A Árvore Binária de Busca (BST) simples mantém um crescimento logarítmico, atingindo uma altura de um pouco mais que 540 para $N = 100.000$. As árvores balanceadas apresentam

alturas mais contidas e próximas ao limite teórico ótimo, com a AVL sendo ligeiramente mais compacta (altura ≈ 19) que a Árvore Rubro-Negra LLRB (altura ≈ 23).

No entanto, o comportamento muda drasticamente nos cenários adversariais (entradas ordenadas, ao centro e à direita). A altura da BST escala linearmente com N , atingindo o valor de 100.000 para a instância máxima, degenerando-se em uma lista simplesmente encadeada. Em contrapartida, tanto a AVL quanto a LLRB detectam o desbalanceamento e mantêm suas alturas rigidamente presas ao limite logarítmico.

4.2 Custo Estrutural: Quantidade de Rotações

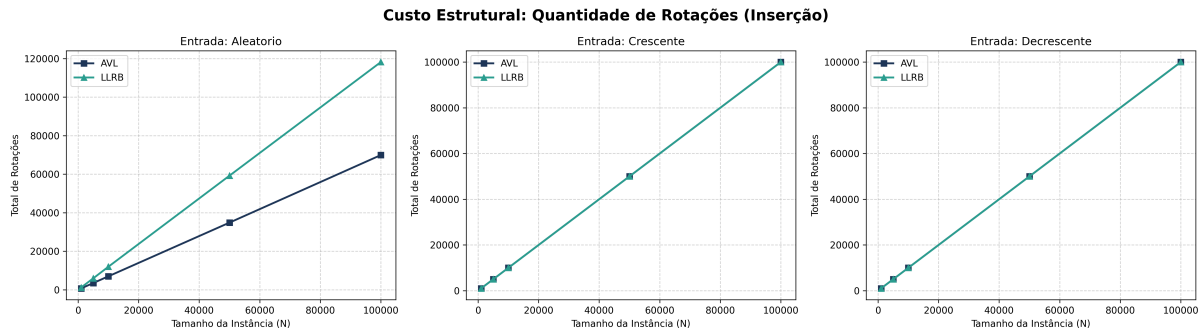


Figura 2: Custo Estrutural: Quantidade de Rotações durante a Inserção

A garantia de compactação das árvores balanceadas exige um custo estrutural em rotações, conforme ilustrado na Figura 2 (a BST possui 0 rotações por definição). No cenário aleatório, a LLRB realizou significativamente mais rotações (aproximadamente 120.000 para a maior instância) do que a AVL (cerca de 70.000).

Contudo, ao enfrentar entradas ordenadas, ambas as estruturas balanceadas realizam um número idêntico de rotações (exatas 100.000 rotações para $N = 100.000$). Esse comportamento evidencia que o esforço estrutural para conter a degeneração da árvore em pior caso é o mesmo para as duas heurísticas de balanceamento.

4.3 Tempo de Execução: Inserção

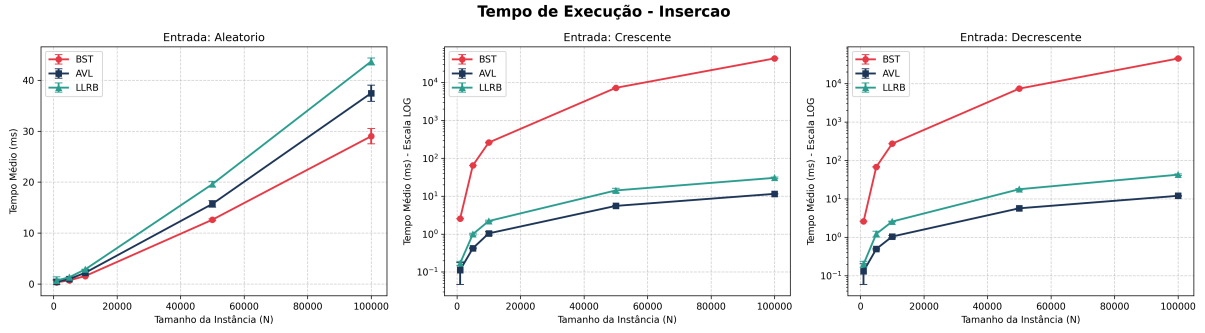


Figura 3: Tempo de Execução - Inserção

O custo temporal das inserções (Figura 3) reflete diretamente a manutenção estrutural. No caso aleatório, a BST apresenta o menor tempo de execução absoluto (≈ 30 ms para $N = 100.000$) devido à ausência de rotações. A AVL atinge ≈ 38 ms, e a LLRB desponta como a mais custosa (≈ 45 ms), reflexo do maior número de rotações observado no gráfico anterior.

Já nos casos ordenados, o impacto da degradação da BST é catastrófico, o que exigiu o uso de escala logarítmica (LOG) no eixo Y. Ao inserir sequencialmente, a BST passa a exigir a travessia de todos os elementos anteriores, operando em tempo $O(N^2)$ e saltando para a ordem de dezenas de segundos ($> 10^4$ ms). Neste cenário de estresse máximo, a AVL e a LLRB entregam excelente performance, com tempos quase idênticos e estabilizados na ordem de 10 a 20 ms.

4.4 Tempo de Execução: Busca

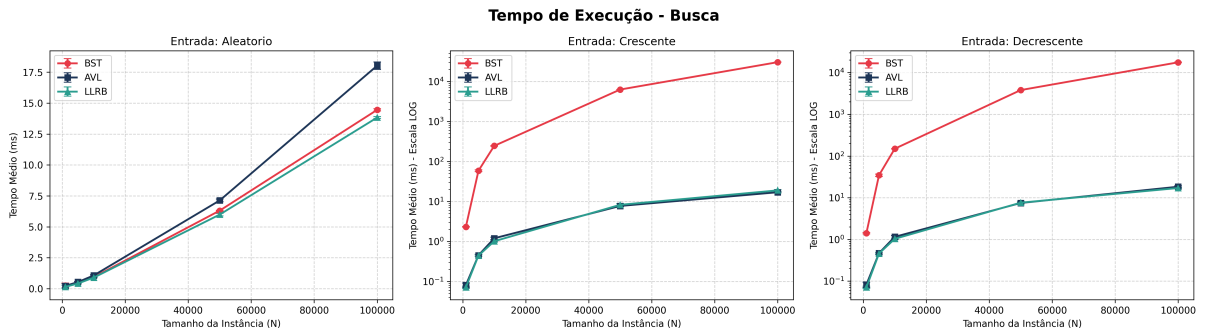


Figura 4: Tempo de Execução - Busca

Durante as buscas puras (Figura 4), onde não há alterações estruturais, a eficiência da busca binária dita as regras. No cenário aleatório, todas as três estruturas performam de

maneira altamente eficiente e equiparável, concluindo as operações em menos de 20 ms.

O cenário adversarial, porém, expõe a vulnerabilidade da BST. Enquanto a AVL e a LLRB continuam resolvendo as buscas em menos de 20 ms através de caminhos proporcionais a $O(\log N)$, a BST exige tempos superiores a 10.000 ms. Isso ocorre porque cada busca na BST degenerada atua com complexidade $O(N)$, percorrendo a estrutura de lista encadeada.

4.5 Tempo de Execução: Remoção

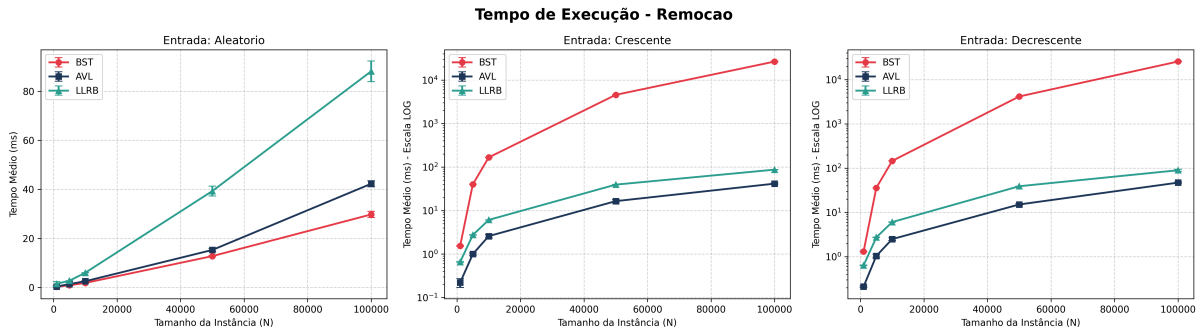


Figura 5: Tempo de Execução - Remoção

A operação de remoção (Figura 5) apresenta nuances interessantes no caso aleatório. O custo de realocação de ponteiros e rebalanceamento torna a LLRB visivelmente mais lenta (≈ 90 ms) em comparação à AVL (≈ 40 ms) e à BST (≈ 30 ms).

Nos casos ordenados, a anomalia temporal se repete: a remoção na BST colapsa para a casa dos milhares de milissegundos devido à necessidade de encontrar o nó na estrutura linear, enquanto a AVL e a LLRB demonstram resiliência, mantendo a eficiência algorítmica constante e estável, independentemente da ordenação prévia dos dados de entrada.

5 Discussão

A análise empírica das três estruturas revelou comportamentos de desempenho que corroboram perfeitamente com a complexidade teórica vista em sala. O uso de instâncias adversariais (dados ordenados) cumpriu o papel fundamental de expor as vulnerabilidades da estrutura não balanceada em seu pior caso, enquanto as entradas aleatórias permitiram avaliar o custo-benefício de cada heurística de balanceamento no caso médio.

5.1 Análise da Árvore Binária de Busca (BST)

A Árvore Binária de Busca (BST) apresentou um desempenho temporal altamente satisfatório no cenário aleatório. Como a ordem de chegada dos elementos é estocástica, a árvore ramifica-se de forma razoavelmente simétrica, mantendo uma altura aceitável e garantindo operações de inserção, busca e remoção muito rápidas, aproximando-se do desempenho $O(\log N)$. Por não possuir sobrecarga com verificações estruturais ou rotações computacionalmente custosas, a BST obteve os menores tempos absolutos de inserção e remoção neste cenário.

No entanto, a submissão a instâncias previamente ordenadas evidenciou sua falha arquitetônica crítica: a total ausência de mecanismos de rebalanceamento automáticos. A árvore degenerou-se em uma lista simplesmente encadeada, elevando sua altura para N . Isso degradou drasticamente a complexidade de busca e remoção para $O(N)$, e converteu a rotina de inserção sequencial de N elementos em um processo de complexidade quadrática $O(N^2)$. Fica evidente empiricamente que a BST clássica é inviável para aplicações de software onde a estrita aleatoriedade da entrada não pode ser garantida.

5.2 Análise da Árvore AVL

A árvore AVL provou ser a estrutura com o controle de altura mais rigoroso entre as implementações testadas, garantindo através do seu Fator de Balanceamento que a diferença de altura entre as subárvores de qualquer nó nunca seja superior a 1. Isso refletiu diretamente na métrica estrutural observada, onde a AVL manteve-se consistentemente como a árvore mais compacta.

No cenário médio (aleatório), a AVL demonstrou um excelente equilíbrio operacional: realizou significativamente menos rotações que a LLRB e obteve tempos de execução menores nas operações de inserção e remoção. Nos cenários adversariais de estresse máximo, a estrutura reagiu instantaneamente à degradação, aplicando rotações constantes que contiveram o crescimento linear. Com isso, fixou o tempo de execução no limite logarítmico estrito, resolvendo em meros milissegundos o processamento que levou dezenas de segundos na árvore não balanceada.

5.3 Análise da Árvore Rubro-Negra LLRB

A Árvore Rubro-Negra com Inclinação à Esquerda (LLRB) atingiu com êxito o objetivo primordial de limitar a altura máxima da estrutura, impedindo o colapso algorítmico nos casos adversariais. Em entradas ordenadas, a LLRB apresentou extrema robustez, equiparando-se ao excelente desempenho temporal e quantidade de rotações da AVL.

Entretanto, a análise do cenário aleatório revelou que a LLRB possui o maior custo de manutenção estrutural entre as árvores avaliadas. A obrigatoriedade de manter a invariante da inclinação à esquerda (onde links vermelhos apontam obrigatoriamente para a esquerda) aliada às propagações de trocas de cores forçaram o algoritmo a realizar consideravelmente mais rotações do que a AVL no caso médio. Esse esforço computacional extra traduziu-se nos gráficos de tempo, tornando a LLRB a estrutura mais lenta para inserções e remoções sob dados aleatórios. Embora a lógica da LLRB resulte em uma implementação de código mais concisa em relação à Rubro-Negra clássica, a abstração cobra o seu preço em sobrecarga operacional.

6 Conclusão

Os resultados experimentais documentados confirmam rigorosamente os fundamentos teóricos de estruturas de dados. A simplicidade algorítmica da BST a torna a opção mais veloz estritamente em ambientes controlados onde a aleatoriedade dos dados é uma premissa inquebrável. Em contrapartida, as estruturas balanceadas justificam plenamente a complexidade extra de suas implementações: o custo computacional residual gasto em rotações e manutenções de invariantes é ínfimo quando comparado ao colapso de performance evitado no pior caso.

Neste experimento de conectividade dinâmica, a árvore AVL destacou-se como a abordagem mais eficiente para a manutenção dos dados. Ao apresentar a menor altura, menor quantidade de rotações no caso médio e tempos de execução consistentemente rápidos e previsíveis, a AVL obteve uma leve, porém clara, superioridade empírica frente à variante LLRB testada.

7 Uso de ferramentas de IA

Conforme os requisitos do trabalho e a Portaria N° 39/PRPPG/UFC, declaramos abaixo o uso de ferramentas de Inteligência Artificial Generativa como apoio ao desenvolvimento deste relatório:

- **Quais ferramentas foram utilizadas:** Gemini
- **Para quais finalidades:** Geração do template em LaTeX, apoio no ajuste do script para a geração dos gráficos.
- **Prompts utilizados:**
 - Gere um template em Latex para o relatório desse trabalho, de acordo com os requisitos de entrega do professor: (Foi passado o texto ao relatório contido no docuemnto de instruções do trabalho disponibilizado no SIGAA)
 - Dado esse código que usa como base o arquivo 'resultados.csv' e gera gráficos de estatísticas, como poderíamos facilitar a leitura desses gráficos. Me ajude a colocar cores que destacam pontos principais e a melhorar a distribuição/leitura deles. (Foi anexado o código do script feito pelo alunos antes das sugestões da IA)

Referências

- [1] ADELSON-VELSKY, G. M.; LANDIS, E. M. (1962). *An algorithm for the organization of information*. Soviet Mathematics Doklady, v. 3, p. 1259-1263.
- [2] BAYER, R. (1972). *Symmetric binary B-Trees: Data structure and maintenance algorithms*. Acta Informatica, v. 1, n. 4, p. 290-306. Springer-Verlag.
- [3] GUIBAS, L. J.; SEDGEWICK, R. (1978). *A dichromatic framework for balanced trees*. In: 19th Annual Symposium on Foundations of Computer Science (SFCS 1978), p. 8-21. IEEE.
- [4] HIBBARD, T. N. (1962). *Some combinatorial properties of certain trees with applications to searching and sorting*. Journal of the ACM (JACM), v. 9, n. 1, p. 13-28. ACM.

- [5] SEDGEWICK, R.; WAYNE, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- [6] HAPPY CODERS. *AVL Tree in Java*. Disponível em: <https://www.happycoders.eu/algorithms/avl-tree-java/>.
- [7] W3SCHOOLS. *DSA AVL Trees*. Disponível em: https://www.w3schools.com/dsa/dsa_data_avltrees.php.